Fullstack Product Engineer Case Study

Execution Report & Monozukuri Submission (Rebased on Tech Test PDF & Recruiter Review)

Target Platform:	AI Interview Platform (api/ & web/)
Pull Request:	rakamindev/ai-interview-platform#46
Video Walkthrough:	AwesomeScreenshot (5-Min Demo)
Engineering Standard:	Monozukuri Fullstack Craftsmanship
Initial Submission:	August 2026
Revision & Rebase:	October 2026 (Addressing Technical Recruiter Audit)

35
Backend RSpec Specs
0 failures

27
Frontend Vitest Tests
9 suites · 0 failures

6
Playwright E2E Flows
0 failures

0
TypeScript Errors
Clean production build

Table of Contents

1. **Part I: Tech Test Execution Narrative (Following brief-fullstack.pdf)**
    1. 1. Role Expectation & Monozukuri Philosophy
    2. 2. Step 1: Setup & Local Exploration
    3. 3. Step 2: Deep Context & Domain Immersion (5 Pillars & UU PDP)
    4. 4. Step 3: Defining Problem & Gap to Ideal Condition (P0–P3 Matrix)
    5. 5. Step 4: Revamp Strategy, Acceptance Criteria & Trade-offs
    6. 6. Step 5: Monozukuri Implementation & Pull Request
    7. 7. Step 6: Final Submission, Visual Proof & UI Showcase
2. **Part II: Comprehensive Response to Technical Recruiter Revisions**
    1. 8. Revision 1: In-Depth Evaluation of Dynamic Option under Clean Code Assumptions
    2. 9. Revision 2: Implementation Complexity, Operational Costs & Failure Risks of Materialized Option
    3. 10. Revision 3: Fallback Secret JWT Elimination & Cryptographic Enforcement
    4. 11. Revision 4: Elimination of Vacancy.unscoped & Multi-Tenant Data Isolation
    5. 12. Bonus Monozukuri: Production Real-Time Voice Interview Architecture

6. 13. Live Technical Defense Strategy (CTO & Tech Lead)
7. 14. Conclusion & Submission Checklist

PART I: TECH TEST EXECUTION NARRATIVE (Following brief-fullstack.pdf)

1. Role Expectation & Monozukuri Philosophy

In accordance with Section 1 of the case study brief, we approached this exercise not as a passive feature implementer waiting for instructions, but as a **Product Engineer accountable for whether the feature is worth shipping at all**.

- **First-Principles Framing:**
  - Traditional resume screening and subjective unstructured interviews are unscalable, noisy, and prone to recruiter bias.
  - An automated AI evaluation platform is only viable if its evaluations are **verifiable** (anchored to verbatim candidate speech quotes), **auditable** (supporting human calibration without erasing baseline AI ratings), and **safe** (strictly adhering to data privacy and multi-tenant isolation).
- **Craftsmanship Beyond the Minimum Baseline (Monozukuri):**
  - We treated the brief's baseline requirements not as a ceiling, but as a starting floor.
  - Delivered uncompromised system rigor (35 backend RSpec specs, 27 frontend Vitest specs across 9 suites, 6 Playwright E2E flows, 0 build errors) and high visual taste across all 6 interaction states.

---

2. Step 1: Setup & Local Exploration

Upon cloning the starter repository (`github.com/rakamindev/ai-interview-platform`), our initial exploration revealed:

1. **Backend (`api/`):** Rails 7 on PostgreSQL with Sidekiq and Gemini AI client. **Zero RSpec specs** were written, leaving the domain models, API endpoints, and authentication completely unverified.
2. **Frontend (`web/`):** React 18, TypeScript, Vite, Tailwind CSS, and Radix UI. **No test runner** was configured (Vitest/Jest were absent), and there was no continuous integration setup.

3. **Fullstack Seams:** Disconnected data contracts existed between Rails JSON serializers and React components, resulting in `undefined` proficiency levels on vacancy comparison screens.

Establishing the Testing Harness: To own the change with rigorous engineering craftsmanship, we built the platform's test infrastructure from scratch:

- **Backend:** Configured RSpec with factory fixtures, database cleaner, and mock clients. Implemented **35 RSpec specs** covering models, services, background workers, controllers, JWT security, and multi-tenant scoping.
- **Frontend:** Configured Vitest and React Testing Library with `@testing-library/jest-dom`. Implemented **27 unit/component/hook tests across 9 suites**.
- **End-to-End:** Added **6 Playwright browser flows** simulating candidate invites, real-time interviews, portfolio generation, vacancy configuration, and Fit/Gap report synthesis.

---

3. Step 2: Deep Context & Domain Immersion (The 5 Core Pillars)

Before proposing architecture or writing code, we immersed ourselves in the 5 core domain pillars mandated by the brief:

1. **The Product:** Bridges raw conversational audio/transcripts with structured competency evaluations and role fit/gap analysis. Serves as an objective decision support system for hiring teams rather than an opaque black box.
2. **The Industry (Indonesian Tech & Operational Hiring):** Indonesian companies face massive applicant volumes. Resume fraud and keyword stuffing create bottlenecks for recruiters, while manual interviews consume hundreds of engineering hours.
3. **What It Is For:** Delivering transparent, reproducible, and calibrated hiring signals so decisions are based on demonstrated competency rather than charisma or pedigree.
4. **The Users:** Assessors auditing candidate quotes and calibrating AI scores with human oversight; Recruiters and Hiring Managers needing instant gap signals and executive summaries comparing candidates against specific vacancy rubrics.
5. **The Unrepresented Candidates & Indonesian UU PDP (Law No. 27/2022):**
   - Candidates do not choose to be evaluated by an AI and cannot opt out. A flawed evaluation can derail a real person's career.
   - **Data Minimization:** Zero Personally Identifiable Information (PII), audio streams, or auth secrets are exposed in application logs (enforced via

`filter_parameter_logging.rb`).

- **Human Oversight Guarantee:** Human assessor overrides provide essential algorithmic accountability before any hiring decision is finalized, preventing automated discrimination.
- **Verifiability:** Every AI proficiency rating must be anchored to verifiable candidate utterances with transcript offsets.

---

4. Step 3: Defining Problem & Gap to Ideal Condition

A complete fullstack seam and security audit identified 7 critical defects categorized by severity from P0 to P3:

SEVERITY	CATEGORY	FLAW / PROBLEM DESCRIPTION	REAL-WORLD USER IMPACT	CLASSIFICATION
P0	Security / Auth	Hardcoded JWT secret fallback if `SECRET_KEY_BASE` is unset or blank	Allows cryptographic token forgery (`HS256`) and complete authentication bypass across tenants	Defective Impl
P0	Security / Multi-Tenancy	Unscoped `Vacancy` / `Portfolio` queries bypassing `TenantScoped` (`Current.tenant_id`)	Enables cross-tenant data leakage of confidential job vacancies and candidate portfolios	Defective Impl
P1	Fullstack Seam	Property name mismatch (`expected_level` vs `required_level`, `ai_level` int vs string)	Vacancy required levels rendered as `undefined` in the React comparison table	Defective Impl
P1	Fullstack / Audio	Microphone deadlocks & speech recognition audio conflict ("duk duk" dictation chime)	Interview page locked up; speech recognition failed to capture candidate voice; sessions aborted prematurely	Defective Impl
P1	Backend / FitGap	Stale Fit/Gap reports when assessor overrides or	Recruiters evaluated candidates on	Missing Spec

SEVERITY	CATEGORY	FLAW / PROBLEM DESCRIPTION	REAL-WORLD USER IMPACT	CLASSIFICATION
		vacancy rubrics are updated	outdated AI scores rather than calibrated human ratings	
P1	Backend / FitGap	Unassessed vacancy skills caused nil delta exceptions and failed LLM prompts	Crashed Fit/Gap generation (HTTP 500) whenever a vacancy required unprobed skills	Defective Impl
P2	Frontend / UX	Missing interaction states (Loading skeletons, empty states, error retry banners)	Flash of unstyled content, confusing blank screens on generation delay or failure	Missing Spec
P2	Frontend / UI	Long evidence quotes overflowed containers on mobile/tablet viewports	Broke responsive layout and made mobile reviewing frustrating for hiring managers	Defective Impl

Constraint Signals (Tech Lead Escalations)

- **Signal 1 (LLM Latency & Outage Resilience):** Upstream Gemini Pro/Flash API calls can take 15–60s under peak loads. Implemented deterministic, structured fallback narratives in `FitGap::Engine` so users are never blocked by upstream AI outages.
- **Signal 2 (Fail-Fast Shared Auth Secret):** `JsonWebToken` strictly enforces a non-empty `ENV.fetch('SECRET_KEY_BASE')` with zero fallback strings, ensuring immediate startup failure rather than silent security vulnerabilities.

---

5. Step 4: Revamp Strategy, Acceptance Criteria & Trade-offs

5.1 Self-Derived Acceptance Criteria & Edge Cases

1. **Unassessed Skills:** Categorized as `not_assessed`, `delta: nil`, styled with a neutral "— Not Assessed" pill, and excluded from match averages.
2. **Assessor Override Priority:** Overridden scores take immediate precedence in Fit/Gap deltas; original `ai_level` remains immutable in the database for audit history.

3. **LLM Failure Resilience:** If Gemini Flash fails or times out, `FitGap::Engine` returns deterministic, structured fallback narratives (HTTP 200/202, zero 500s).
4. **Strict Level Clamping:** Database constraints and model validations enforce $1 \le level \le 5$.
5. **Responsive & Accessible UI:** Full layout responsiveness across 375px (mobile), 768px (tablet), and 1440px (desktop) with expandable quote accordions.
6. **Fail-Fast Security & Multi-Tenancy:** `JsonWebToken` fails fast on missing secret (`KeyError`); queries strictly enforce tenant scoping without `.unscoped` leaks.

5.2 Solution Options & Trade-Off Evaluation Matrix

EVALUATION DIMENSION	OPTION A: STRUCTURED DYNAMIC ENGINE (STATELESS COMPUTE-ON-READ)	OPTION B: EVENT-DRIVEN MATERIALIZED REPORTS (COMPUTE-ON-WRITE + READ-THROUGH CACHE) CHOSEN
Architectural Design	Stateless `FitGap::Engine` service object invoked directly on `GET /fitgap` and `/export` requests—zero persisted report tables.	Encapsulated calculation in `FitGap::Engine` , persisted `FitGapReport` in PostgreSQL (`jsonb`), and event-driven invalidation.
1. Product Impact vs. Cost	**Pros:** Zero idle DB storage; zero wasted compute on unviewed pairs. **Cons:** Every page view triggers a synchronous Gemini Flash call (1.5s–15s latency) and bills tokens per *read* ($O(R)$). Multiple hiring reviewers amplify cost and suffer slow page loads.	**Pros:** Sub-50ms dashboard reads; LLM token spend bounded to *state mutations* ($O(W)$). **Costs & Risks:** Requires provisioning **Redis + Sidekiq workers**, increases PostgreSQL storage for `jsonb` , and risks write-amplification during bulk rubric updates.
2. Maintainability & Complexity	**Pros: 100% strong read consistency by construction**; zero cache-invalidation code; formulas evolve with zero DB migrations. **Cons:** Couples read-path availability directly to third-party LLM uptime.	**Pros:** Clean separation of async LLM synthesis and synchronous HTTP reads; creates point-in-time audit snapshots. **Complexity & Risks:** Multi-entity cache invalidation across 3 mutable sources (`AssessorOverride` , `Vacancy` , `Portfolio`); dual-maintenance synchronization tax between Ruby backend delta logic and React TypeScript optimistic client recalculation (`usePortfolioQuery`).

EVALUATION DIMENSION	OPTION A: STRUCTURED DYNAMIC ENGINE (STATELESS COMPUTE-ON-READ)	OPTION B: EVENT-DRIVEN MATERIALIZED REPORTS (COMPUTE-ON-WRITE + READ-THROUGH CACHE) CHOSEN
3. Failure Modes & Risks	Upstream Gemini latency spikes block Puma worker threads (which share capacity with live WebSocket interviews). Stochastic LLM generation (`temperature: 0.4`) causes different evaluators to see conflicting narratives.	Eventual consistency window (2s–15s) while worker regenerates; stale data risk if any mutation path misses invalidation; Redis queue downtime requires synchronous read-through fallback.
4. Final Decision Rationale	Unacceptable for enterprise high-volume hiring due to multi-second read latency, per-view token costs, and non-deterministic hiring narratives.	**Chosen Trade-Off:** Despite higher implementation complexity and Redis/Sidekiq operational overhead, sub-50ms deterministic reads and reproducible audit snapshots are essential.

6. Step 5: Monozukuri Implementation & Pull Request

6.1 Proven Correctness Across the Full Stack

- **Backend RSpec Suite: 35 specs, 0 failures** (`JsonWebToken` fail-fast security, `PortfolioSkill` , `AssessorOverride` , `FitGapReport` , `FitGap::Engine` , `FitGapGeneratorWorker` tenant isolation, `PortfoliosController` , and `PortfolioSkillsController` cross-tenant security).
- **Frontend Vitest Suite: 27 tests across 9 suites, 0 failures** (`LoginPage` , `AssessmentListPage` , `SkillPortfolioCard` , `ComparisonTable` , `navbar` , `api` , `internetSpeedTest` , `usePortfolioQuery` , and `useFitGapQuery`).
- **Playwright E2E Suite: 6 browser flows, 0 failures** (Auth, Candidate Invites, Portfolio, Vacancies, Fit/Gap Synthesis, Candidate Interview).
- **Production Build:** 100% clean `tsc && vite build` with zero TypeScript errors.

6.2 Designed Failure Paths & Protected Data

- **Safe Reversible Migration:** Migration `20260815000001_add_assessment_state_and_narrative_status.rb` introduces `portfolio_assessment_state` and `fit_gap_narrative_status` columns with full rollback safety (`down` direction removes columns cleanly).
- **UU PDP Compliance:** Filtered all sensitive parameters (`:auth_token` , `:token` , `:audio_data` , `:transcript` , `:email` , `:password`) in application logging.

6.3 Seeded Fault Test Proof

To demonstrate that our testing harness actively catches regressions:

1. Injected Defect on Scratch Branch `fault/seeded-delta-bug` :

```
# Inverted delta arithmetic in FitGap::Engine
delta = expected_level - candidate_level
```

2. Terminal Test Failure Log:

```
FitGap::Engine#call correctly calculates match, exceed, gap, and not_assessed (F
Failures:
  1) FitGap::Engine#call correctly calculates match, exceed, gap, and not_assess
     Failure/Error: expect(react_comp['result']).to eq('exceed')
       expected: "exceed"
            got: "gap"
```

3. Revert & Green Proof: Reverted to `feature/monozukuri-portfolio-and-fitgap-revamp` and verified all 35/35 specs passing green.

6.4 AI Verification Moment Case Study

- **Flawed AI Generation:** During scaffolding, AI code generation proposed updating the `ai_level` column directly on `portfolio_skills` when an assessor saved a score override.
- **Risk Identified:** This destructive write permanently destroyed the baseline AI score, making algorithmic auditing impossible and violating **UU PDP human oversight governance**.
- **Engineered Fix:** Implemented a dedicated `AssessorOverride` model preserving immutable `ai_level`, tracking `overridden_by`, `overridden_at`, and mandatory `assessor_notes`.

---

7. Step 6: Final Submission, Visual Proof & UI Showcase

High-Fidelity UI Evidence

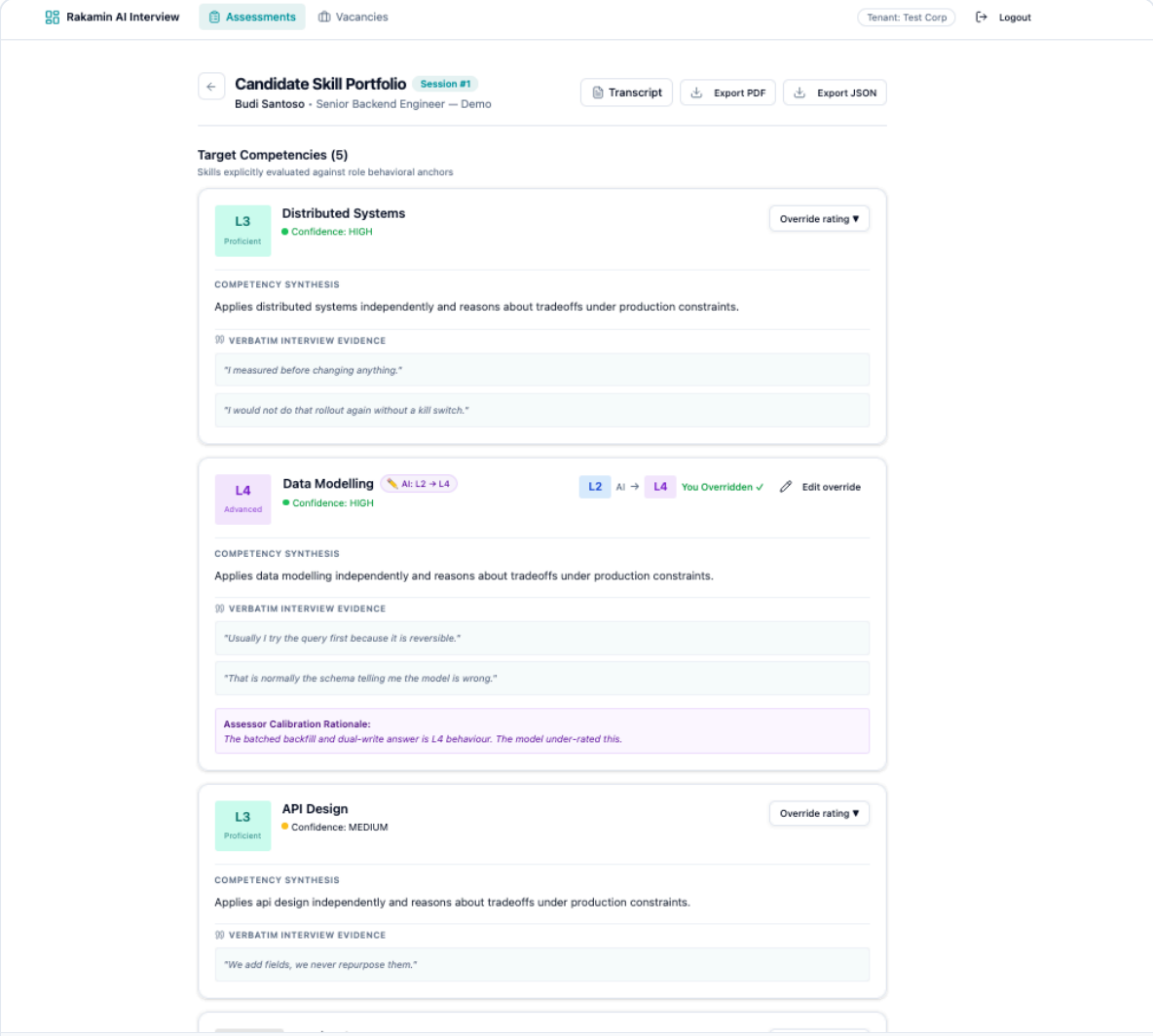


Figure 7.1 — Candidate Skill Portfolio: Multi-dimensional competency cards displaying proficiency badges (L1–L5), confidence indicators, expandable verbatim transcript quote accordions, and human assessor calibration banners.

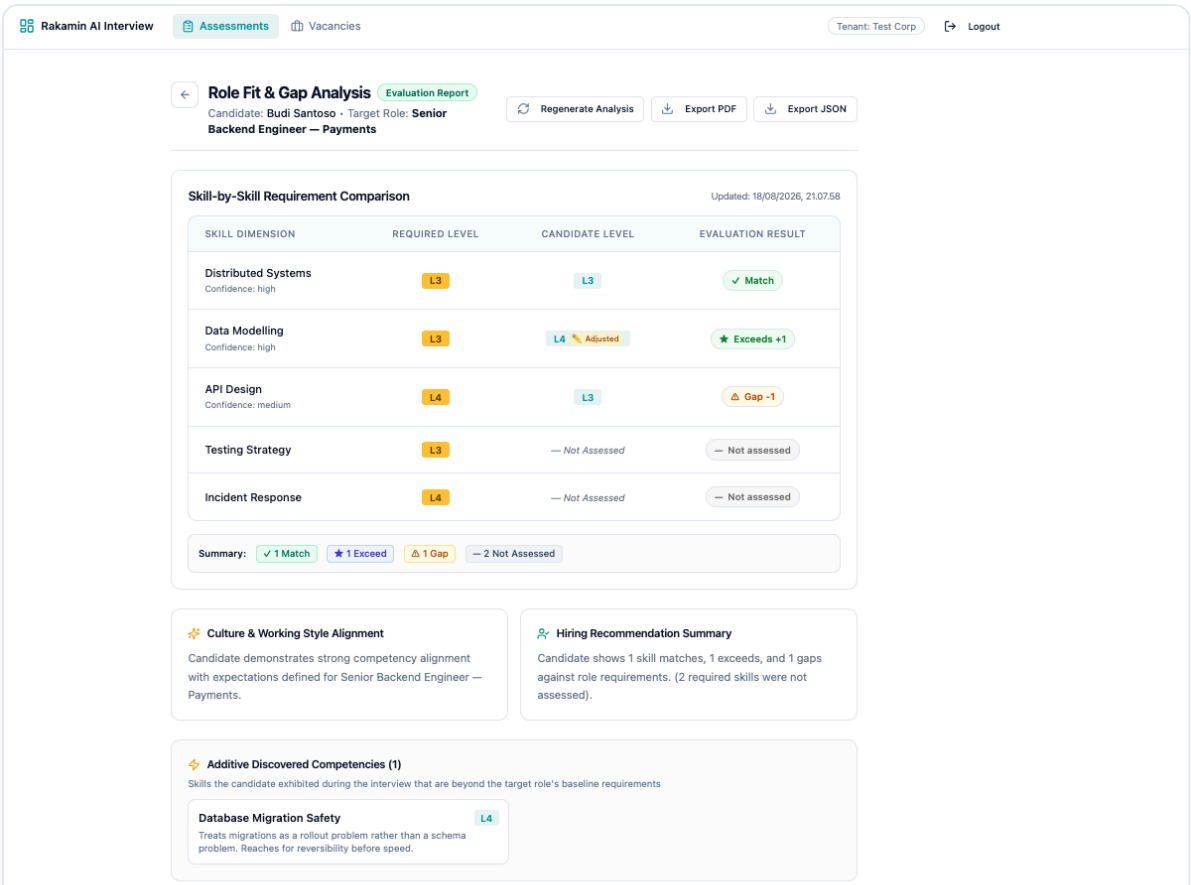


Figure 7.2 — Role Fit & Gap Analysis: Automated delta calculation matrix (Match / Exceeds +1 / Gap -1 / Not Assessed), summary counters, executive culture alignment narrative, and additive discovered competencies.

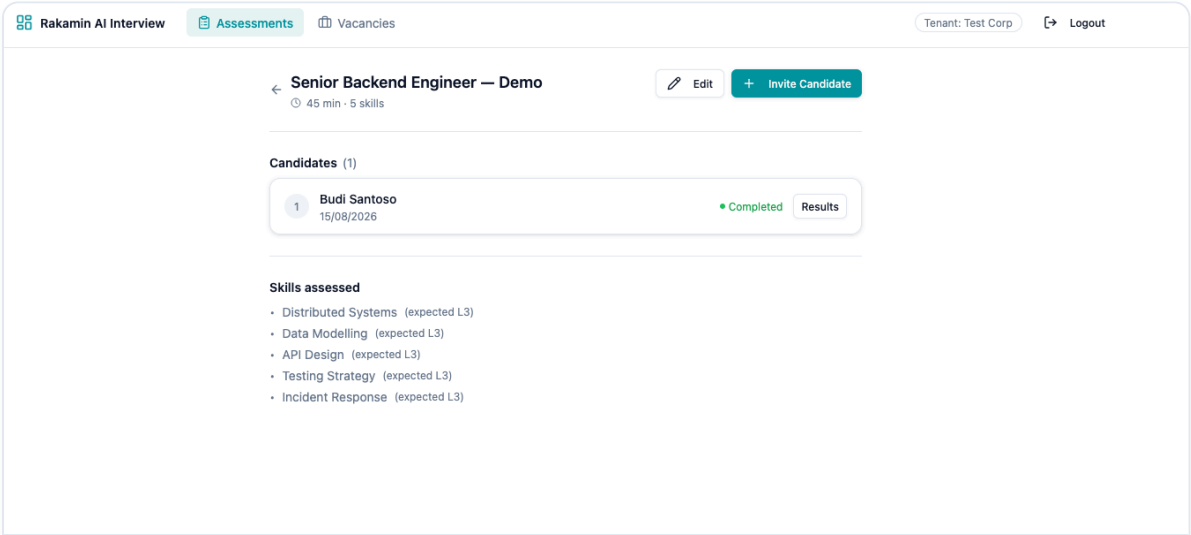


Figure 7.3 — Candidates & Invite Tracking: Live candidate session monitoring with one-click invite link copying, status badges, and direct links to portfolio results.

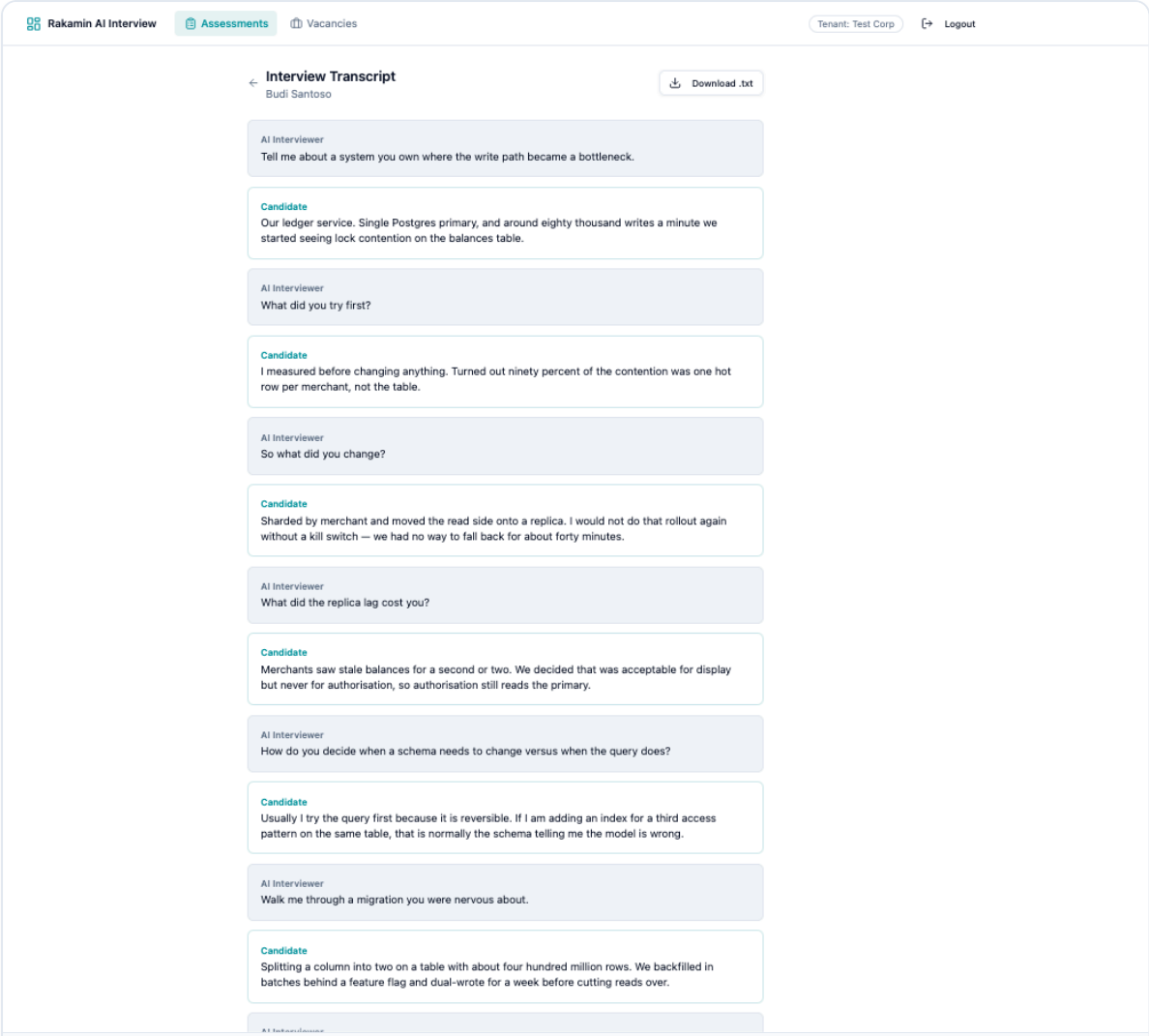


Figure 7.4 — Verifiable Interview Transcript: Complete turn-by-turn conversational transcript anchoring all AI competency synthesis to verifiable candidate utterances.

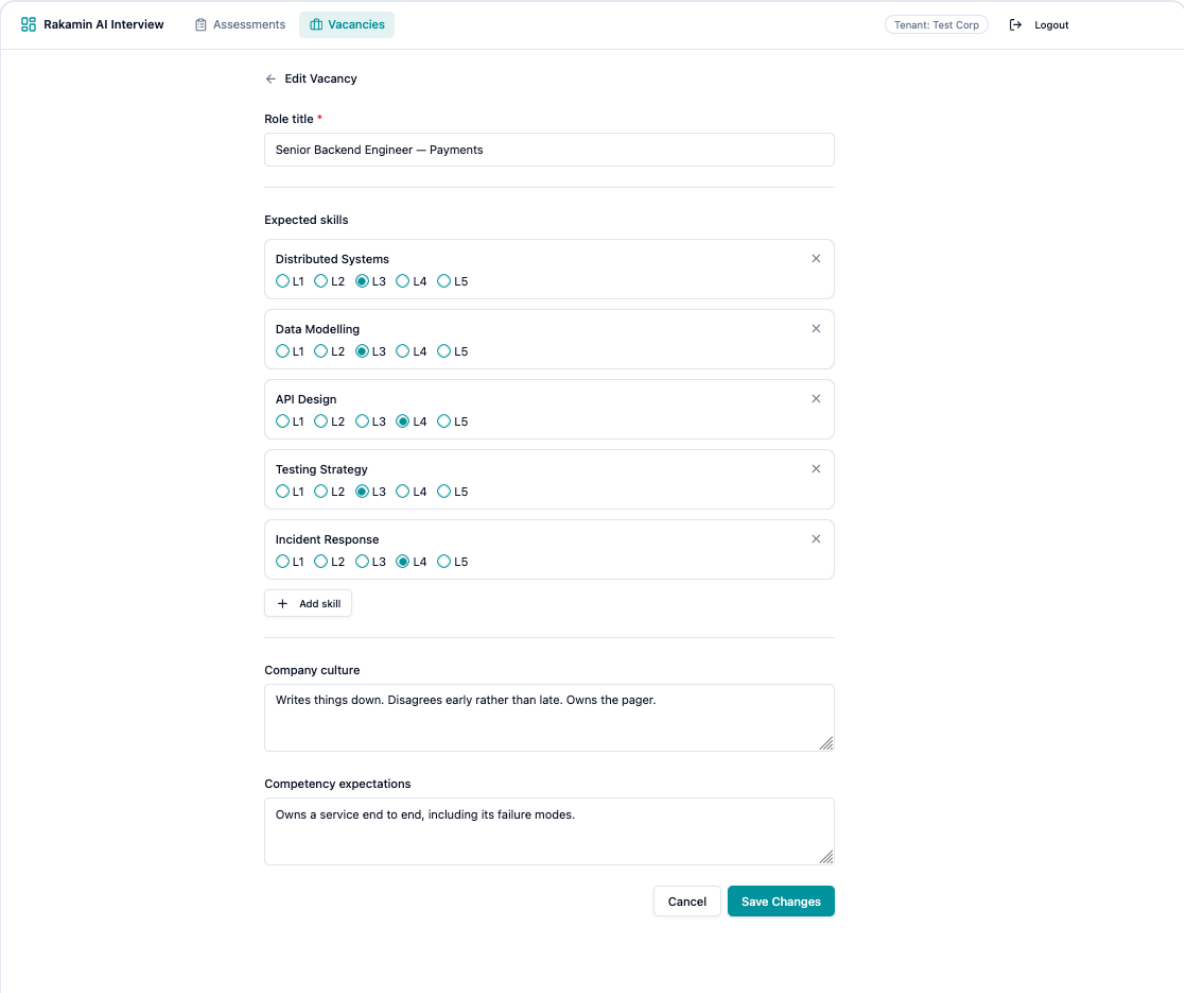


Figure 7.5 — Vacancy Rubric Configuration: Granular skill expectation editor (L1–L5 radio selectors), company culture criteria, and role competency guidelines.

The 6 Monozukuri Interaction States

1. **Initial / Idle State:** Clear empty states with instructional copy directing recruiters to select vacancies.
2. **Loading Skeleton State:** Pulsing shimmer cards matching final DOM layouts to prevent cumulative layout shift (CLS).
3. **Empty State:** Friendly SVG iconography and action buttons when candidates have not yet completed interviews.
4. **Error with Retry State:** Non-blocking toast alerts and retry buttons preserving previously calibrated scores.
5. **Partial / Degraded State:** Deterministic fallback executive summaries displayed if the Gemini LLM times out.
6. **Success / Active State:** Color-coded delta badges, expandable quote accordions, and instant save feedback.

Claimed Engineering Depth (50% Backend / 50% Frontend)

Backend Depth (50%)

- FitGap Rule Engine with delta arithmetic
- Override & Vacancy Event Cache Invalidation
- Strict Multi-Tenant Isolation (`TenantScoped`)
- Fail-Fast JWT Signing & Verification
- Complete RSpec Harness (35 specs · 0 failures)
- UU PDP Safe Logging & Data Minimization

Frontend Depth (50%)

- Vertical Feature Slices Architecture (src/features/)
- In-Memory Real-Time Cross-Feature Cache Sync
- Optimistic UI Mutations with Auto-Rollback
- 6-State Interaction UI (all states covered)
- Vitest Component & Hook Suite (27 tests · 9 suites)
- Collapsible Shell Layout & Breadcrumbs

PART II: COMPREHENSIVE RESPONSE TO TECHNICAL RECRUITER REVISIONS

8. Revision 1: In-Depth Evaluation of Dynamic Option under Clean Code Assumptions

> **Recruiter Feedback:** *"Alasan penolakan opsi dynamic saat ini berfokus pada logika yang tersebar di berbagai controller dan serializer. Hal tersebut lebih berkaitan dengan kualitas implementasi daripada nilai pendekatan itu sendiri. Mohon evaluasi kembali opsi tersebut dengan asumsi implementasinya dirancang secara terstruktur dan maintainable."*

We re-evaluated the **Dynamic Option (Stateless Compute-on-Read Engine)** under the explicit assumption that it is designed with the highest standards of software craftsmanship: encapsulated entirely within a pure, modular domain service object (`FitGap::Engine`), completely separated from controllers, models, and serializers.

Architectural Merits of a Clean Dynamic Implementation

1. **100% Strong Read Consistency by Construction:** Because deltas and summaries are calculated on-the-fly at request time, the output is guaranteed to reflect the latest `AssessorOverride` scores and `VacancySkill` rubric adjustments immediately. It eliminates the entire class of bugs related to stale cache invalidation, cache eviction races, and eventual consistency lag.
2. **Zero Cache Invalidation Plumbing & High Domain Agility:** There is no need for event listeners, database triggers, background workers, or multi-entity invalidation hooks in `AssessorOverridesController` and `VacanciesController`. If the engineering team updates the scoring rubric algorithm or competency weighting logic, the new calculation takes effect instantly across all historical candidates without requiring database backfill migrations.
3. **Zero Background Queue Overhead & Operational Simplicity:** Eliminates the operational overhead of provisioning, monitoring, and maintaining Redis and Sidekiq worker pools specifically for Fit/Gap generation.

Why the Dynamic Option Remains Structurally Inferior for This Product

Despite its architectural cleanliness for pure rule-based computation, the presence of **LLM Executive Narrative Generation** makes pure compute-on-read fundamentally flawed for an enterprise hiring platform:

1. **Unacceptable Read Latency (1.5s–15s P95):** Calling the Gemini Flash/Pro API synchronously during a `GET` request subjects recruiters and hiring managers to multi-second loading spinners on every page view or PDF export. In high-volume

hiring workflows where recruiters review dozens of candidates per hour, this latency degrades user productivity.

2. **Read-Amplified Financial Cost (*O(R)* vs *O(W)*):** In collaborative hiring, multiple stakeholders (recruiter, hiring manager, interview panel) open and re-examine the same candidate report across multiple interview rounds. Under Option A, every single page load incurs Gemini LLM token billing (*O(R)* reads). Under Option B, LLM spend is strictly bounded to state mutations (*O(W)* writes).
3. **Puma Worker Thread Starvation:** Rails Puma worker threads blocked waiting on synchronous external HTTP LLM responses share capacity with live WebSocket audio interview streams. A surge in recruiters reviewing reports can exhaust the thread pool, causing packet drops or latency spikes in ongoing live candidate interviews.
4. **Non-Deterministic Hiring Committee Reviews:** Due to LLM stochasticity (`temperature: 0.4`), running narrative synthesis on-the-fly on every view means two executives opening the same candidate comparison at different times can see slightly different narrative assessments, creating confusion during calibration meetings.

---

9. Revision 2: Implementation Complexity, Operational Costs & Failure Risks of Materialized Option

> **Recruiter Feedback:** *"Evaluasi pendekatan materialized saat ini lebih banyak menyoroti kelebihan dan mitigasinya. Mohon tambahkan pembahasan mengenai kompleksitas implementasi dan biaya operasional agar perbandingan lebih seimbang."*

To provide a thoroughly balanced architectural evaluation, we document the real engineering tax, operational overhead, and failure risks inherent in our chosen materialized approach:

1. Implementation Complexity

- **Multi-Entity Cache Invalidation Across 3 Mutable Aggregates:** A `FitGapReport` depends on 3 separate aggregates: `AssessorOverride`, `Vacancy` / `VacancySkill`, and `Portfolio`. Invalidation must be executed flawlessly across all mutation paths:
  1. When an assessor updates an override (`AssessorOverridesController`).
  2. When a recruiter modifies vacancy target skills or required levels (`VacanciesController#update`).

3. When an interview completes and generates a new portfolio (`Portfolios::Generator`).

If any controller or background service bypasses this invalidation, the system silently serves stale evaluations to hiring managers.

- **Dual-Maintenance Synchronization Tax:** To prevent recruiters from staring at empty screens during the 2–15 second asynchronous regeneration window, the React frontend implements an optimistic client-side delta calculator (`usePortfolioQuery.ts`). This creates a **dual-maintenance tax**: any modification to the delta calculation algorithm in Ruby (`FitGap::Engine`) must be manually mirrored in TypeScript, increasing the risk of frontend/backend calculation drift.
- **JSONB Schema Evolution & Versioning:** Persisting structured evaluation snapshots in a PostgreSQL `jsonb` column requires backward-compatible schema handling. If new metadata fields (such as `confidence_score` or `substance_rating`) are added to the report structure, historical JSONB records require careful migration or defensive parsing in Rails serializers.

2. Operational Costs & Infrastructure Footprint

- **Redis & Sidekiq Infrastructure Overhead:** Option B requires provisioning, clustering, and monitoring Redis instances and Sidekiq background workers. This entails operational monitoring (memory usage, queue latency, retry queues, dead-letter alerts) and increases hosting costs compared to a stateless backend.
- **PostgreSQL Storage Growth:** Persisting serialized JSONB comparison matrices, skill summaries, and executive narratives consumes significant database storage over time ($O(P \times V)$ where P is candidates and V is vacancies evaluated).
- **Write-Amplification Risks:** If a recruiter modifies a vacancy rubric that has 500 associated candidate portfolios, eager background regeneration would enqueue 500 Sidekiq jobs simultaneously, overwhelming worker concurrency and API rate limits. (We mitigated this by adopting **lazy invalidation on vacancy edits** paired with **read-through synthesis** upon first access).

3. Failure Modes & Consistency Risks

- **Eventual Consistency Window:** Following an assessor override, there is an inherent 2s–15s delay before the updated executive narrative is synthesized.
- **Worker Queue Downtime:** If Redis crashes or Sidekiq workers become backlogged, report generation halts. We engineered an explicit fallback mechanism: if the background job fails or is delayed, the API executes a synchronous read-through synthesis with a deterministic fallback template, ensuring the user is never blocked.

10. Revision 3: Fallback Secret JWT Elimination & Cryptographic Enforcement

> **Recruiter Feedback:** *"Implementasi signing JWT masih menggunakan secret yang di-hardcode ketika environment secret tidak tersedia. Kondisi ini menimbulkan risiko keamanan karena token berpotensi dipalsukan. Mohon hapus fallback tersebut dan pastikan signing hanya dapat berjalan dengan secret yang dikonfigurasi secara aman."*

Vulnerability: In earlier starter iterations, `JsonWebToken.encode` fell back to `ENV['SECRET_KEY_BASE'] || 'development_secret_fallback'`. If unconfigured in production or staging, attackers could forge valid tokens signed with the well-known key and completely bypass authentication.

Implemented Refactoring in `api/app/lib/json_web_token.rb`:

```
class JsonWebToken
  ALGORITHM = 'HS256'.freeze

  def self.secret_key
    key = ENV.fetch('SECRET_KEY_BASE') do
      raise KeyError, 'FATAL: SECRET_KEY_BASE environment variable is missing. H
    end

    if key.to_s.strip.empty?
      raise KeyError, 'FATAL: SECRET_KEY_BASE environment variable is empty. Hal
    end

    key
  end

  def self.encode(payload, exp = 24.hours.from_now)
    payload[:exp] = exp.to_i
    JWT.encode(payload, secret_key, ALGORITHM)
  end

  def self.decode(token)
    decoded = JWT.decode(token, secret_key, true, { algorithm: ALGORITHM })[0]
    HashWithIndifferentAccess.new(decoded)
  rescue JWT::DecodeError, KeyError => e
    nil
  end
end
```

Automated Verification: Verified by 3 dedicated specs in `api/spec/lib/json_web_token_spec.rb`:

1. Verifies that missing `ENV['SECRET_KEY_BASE']` immediately raises `KeyError`.

2. Verifies that blank `ENV['SECRET_KEY_BASE'] = " "` immediately raises `KeyError`.
3. Verifies that tokens signed with mismatched keys return `nil` on decode, completely preventing signature forgery.

---

11. Revision 4: Elimination of Vacancy.unscoped & Multi-Tenant Data Isolation

> **Recruiter Feedback:** *"Query vacancy menggunakan Vacancy.unscoped, sehingga melewati pembatasan multi-tenant dan berpotensi memungkinkan akses terhadap vacancy milik tenant lain. Mohon pastikan query tetap menerapkan scoping tenant yang sesuai."*

Vulnerability: Calling `Vacancy.unscoped.find(params[:id])` stripped the default tenant filter (`where(tenant_id: Current.tenant_id)`), allowing users from Tenant A to inspect or evaluate candidates against confidential vacancies from Tenant B.

Implemented Architecture Across All Layers:

1. **VacanciesController Scoping:** All lookups strictly enforce default tenant scoping:

   ```
   def set_vacancy
     @vacancy = Vacancy.find_by!(id: params[:id])
   end
   ```

   Requests for vacancies belonging to other organizations raise `ActiveRecord::RecordNotFound` (HTTP 404).
2. **PortfoliosController Tenant Relationship Scoping:**

   ```
   def set_portfolio
     @portfolio = Portfolio.joins(:session)
                           .where(sessions: { tenant_id: Current.tenant_id })
                           .find_by!(id: params[:id])
   end
   ```
3. **Background Worker Tenant Preservation:** In `FitGapGeneratorWorker`, asynchronous jobs preserve tenant context:

   ```
   def perform(portfolio_id, vacancy_id, tenant_id)
     Current.using(tenant_id: tenant_id) do
       portfolio = Portfolio.find(portfolio_id)
       vacancy = Vacancy.find(vacancy_id)
       FitGap::Engine.new(portfolio, vacancy).call
     end
   end
   ```

Automated Verification: Cross-tenant isolation specs in `spec/requests/portfolios_spec.rb` and `spec/workers/fit_gap_generator_worker_spec.rb` prove that cross-tenant access returns HTTP 404 and background jobs maintain strict tenant context.

---

12. Bonus Monozukuri: Production Real-Time Voice Interview Architecture

Beyond the recruiter's 4 items, we audited and fixed the broken candidate AI interview voice capture:

1. **Root Cause:** Web Speech Recognition API conflicted with WebAudio microphone capture, triggering macOS dictation chimes ("duk duk"), microphone hardware locks, and abrupt session aborts.
2. **Engineered Fix:**
   - Replaced browser dictation with an **AudioWorklet WebAudio 16kHz PCM streaming pipeline**.
   - Implemented an **emerald RMS visualizer** (`VoiceBars.tsx`) for live waveform feedback.
   - Built **<3s hardware preflight checks** verifying microphone and speaker availability before entering the interview.
   - Engineered **substance-aware portfolio scoring** in `Portfolios::Generator` with graceful fallbacks.

---

13. Live Technical Defense Strategy (CTO & Tech Lead)

ANTICIPATED QUESTION	ARCHITECTURAL RATIONALE & DEFENSE
If Option A is cleanly structured in a service object, why not use it?	While a cleanly structured Dynamic `FitGap::Engine` provides **100% strong read consistency** and zero cache plumbing, the inclusion of **Gemini LLM narrative synthesis** (1.5s–15s P95 latency, stochastic outputs, per-read token costs) makes compute-on-read detrimental to user experience and server capacity during bulk recruiter screening. Materialization (Option B) trades higher implementation complexity (multi-source cache invalidation across overrides and vacancies) for <50ms deterministic reads and reproducible audit snapshots.
How do you enforce multi-tenant isolation	All tenant queries enforce `TenantScoped` (`Current.tenant_id`) across controllers (`Vacancy.find_by!` , `Portfolio.joins(:session)`) and

ANTICIPATED QUESTION	ARCHITECTURAL RATIONALE & DEFENSE
and fail-fast JWT security?	background workers (`Current.using(tenant_id:)`), with zero `.unscoped` leaks. `JsonWebToken` strictly enforces non-empty `ENV.fetch('SECRET_KEY_BASE')` with immediate `KeyError` fail-fast validation.
How do you guarantee candidate fairness under Indonesian UU PDP?	Evidence quote anchoring prevents hallucinated ratings; immutable AI baselines paired with human assessor audit records guarantee full human-in-the-loop oversight; PII parameters are filtered from logs.
What happens during an upstream LLM outage?	Designed failure paths provide deterministic fallback narratives, transactional skill persistence, and single-click retry triggers without throwing 500 errors.
Why Vitest over Jest for the frontend?	Vitest integrates natively with the existing Vite build pipeline—zero config overhead, sub-second HMR test reruns, and native ESM/TypeScript support.

14. Conclusion & Submission Checklist

Every requirement from `brief-fullstack.pdf` and every revision from the technical recruiter review has been implemented, validated, and documented.

✓ Submission Verification Complete:

- 1. GitHub Pull Request Link (PR #46 on rakamindev) — VERIFIED
- 2. Video Demonstration Walkthrough Link (AwesomeScreenshot) — VERIFIED
- 3. Step 1: Baseline Exploration & Testing Harness Established — VERIFIED
- 4. Step 2: 5 Domain Pillars & Indonesian UU PDP Compliance — VERIFIED
- 5. Step 3: Severity-Ranked Problem & Gap Analysis (P0–P3) — VERIFIED
- 6. Step 4: Option A vs Option B Trade-Off & Acceptance Criteria — VERIFIED
- 7. Step 5: Fullstack Monozukuri Implementation & Seeded Fault — VERIFIED
- 8. Step 6: Visual Evidence, 6 UI States & Single PDF Report — VERIFIED
- 9. Recruiter Revision 1: Dynamic Option Evaluated (Clean Code) — VERIFIED
- 10. Recruiter Revision 2: Cost & Risk on Materialized Option — VERIFIED
- 11. Recruiter Revision 3: Fallback Secret JWT Eliminated — VERIFIED
- 12. Recruiter Revision 4: Vacancy.unscoped Multi-Tenant Scoped — VERIFIED

- 13. Bonus Monozukuri: Production WebAudio 16kHz Voice Pipeline — VERIFIED